Section 1100 In-Class Exercise

Mapping the Best Tacos in Fort Worth with Python and Mapbox

DCDA Capstone Course — In-Class Exercise

Overview

In this exercise you will use Python, Mapbox, and Folium to build an interactive web map of Fort Worth's best taco shops. You will also encounter a critical real-world challenge in web scraping: JavaScript-rendered websites. By the end of class you will have two maps — one with a plain basemap and one with a custom TCU-themed style you design yourself.

You will complete five steps in order:

Step	What You Do
1	Create a free Mapbox account
2	Run Script 1 — attempt to scrape Texas Monthly, learn about JS rendering
3	Run Script 2 — geocode addresses and build a basic map
4	Design your custom TCU-themed Mapbox style
5	Run Script 3 — rebuild the map with your custom style

Before You Begin

1. Open VS Code and open the `1100` folder as your workspace
2. Open a **Terminal**: go to **Terminal** → **New Terminal**
3. Install the required Python libraries:

```
pip install -r requirements.txt
```

Wait for the installation to finish before proceeding.

Step 1: Create Your Mapbox Account

Mapbox is a mapping platform that provides geocoding APIs and a style editor for designing custom maps.

Instructions:

1. Go to https://mapbox.com and click **Sign Up**

2. Fill in your name, email, and a password. Use your school email if possible.
3. Verify your email by clicking the link Mapbox sends you
4. Select the **Free** plan when prompted — no credit card required
5. Explore your dashboard briefly — note the **Studio**, **Tokens**, and **Access tokens** sections

> If you already created a Mapbox account in another section or for a previous assignment, you can use the same account.

Step 2: Run Script 1 — Web Scraping and the JavaScript Problem

This step teaches you something important: **not all websites can be scraped with simple Python tools**. The script will first attempt to pull data from the Texas Monthly "Best Tacos in Fort Worth" article, then explain why it can't — and use a curated backup dataset instead.

What is Web Scraping?

Web scraping means writing code that automatically downloads a webpage and extracts structured data from it. You use the `requests` library to download raw HTML (the code a webpage is built from) and `BeautifulSoup` to find and read specific parts of it.

The JavaScript Problem

Many modern websites — including Texas Monthly — do **not** put their actual content in the raw HTML. Instead, they ship the HTML as an empty container and use **JavaScript** to fill in the content after the page loads. When your Python script downloads such a page, it only gets the empty container — the JavaScript hasn't run yet, so the content isn't there yet.

Here is the difference:

Site Type	What Python's `requests` sees	Can we scrape it easily?
Static HTML site	Full article text, names, addresses	☑ Yes
JavaScript-rendered site (like Texas Monthly)	Empty container or skeleton HTML	❌ Not directly

Script 1 will deliberately show you what happens when you try to scrape a JavaScript site, then fall back to a curated dataset of Fort Worth taco shops.

Run the Script:

1. In the terminal, navigate to the `1100` folder:

```
cd 1100
```

2. Run Script 1:

```
python scripts/01_scrape_fw_tacos.py
```

3. Read the output carefully — notice the explanation about JavaScript rendering
4. When complete, open `data/fw_tacos.csv` in VS Code to explore the data. You should see 12 Fort Worth taco shops with columns for **Name**, **Address**, **Neighborhood**, **Specialty**, and **Description**.

> **Real-world note:** When you encounter a JavaScript-rendered site in your future work, you have two options: (1) use a browser automation tool like **Selenium** or **Playwright** that actually runs a real browser and waits for JS to load, or (2) look for an official API or downloadable data export that the site might offer. For class today, option (2) is our approach — we use curated data.

Step 3: Run Script 2 — Geocode Addresses and Build a Basic Map

Script 2 converts street addresses to GPS coordinates using the Mapbox API, then builds an interactive map.

Part A: Get Your Mapbox Access Token

Your Python scripts need an **access token** to use Mapbox's APIs. This is like a password or API key that identifies your account.

How to find your token:

1. Go to https://account.mapbox.com/access-tokens/
2. Sign in with your Mapbox account
3. Find the **"Default public token"** — it starts with `pk.`
4. Click the copy icon to copy it

> **Security reminder:** Never post your access token to GitHub, social media, or a public file. For this class exercise, pasting it in the script is acceptable. In professional projects, tokens are stored in environment variables or `.env` files that are excluded from version control.

Part B: Add Your Token to Script 2

1. Open `scripts/02_geocode_map_basic.py` in VS Code
2. Find this line:

```
MAPBOX_TOKEN = "PASTE_YOUR_MAPBOX_TOKEN_HERE"
```

3. Replace the placeholder with your real token:

```
MAPBOX_TOKEN = "pk.eyJ1IjoieW91cnVzZXJuYW1lIiwiYSI6ImNsZX..."
```

4. Save the file (**Ctrl+S**)

Part C: Run the Script

```
python scripts/02_geocode_map_basic.py
```

What the script does:

- Loops through each taco shop address and sends it to the Mapbox Geocoding API
- The API returns latitude and longitude for each address
- Saves a new CSV with `Latitude` and `Longitude` columns added
- Creates an interactive Folium map with markers colored by neighborhood
- Saves the map to `data/fw_tacos_map_basic.html`

To view your map:

1. In the VS Code Explorer, find `1100 → data → fw_tacos_map_basic.html`
2. Right-click → **Open with Live Server** (or open in your browser directly)

You should see Fort Worth with taco shop markers color-coded by neighborhood. Click any marker to see its popup with the name, address, neighborhood, specialty, and description. Hover over a marker to see its name.

> **What is Folium?** Folium is a Python library that generates interactive maps using **Leaflet.js** (a popular JavaScript mapping library) under the hood. When you call `folium.Map()` and `folium.Marker()`, Folium writes the HTML and JavaScript code needed for the interactive map and saves it as a standalone `.html` file — no web server required.

Step 4: Design Your Custom Mapbox Style — TCU Color Palette

In this step you will create a map style that tastefully reflects the TCU brand: deep purple, white, charcoal grey, and a very minor red accent. The goal is a polished, professional-looking map that feels distinctly TCU without being garish.

TCU Brand Color Reference

Element	Color Name	Hex Code	Notes
Land (base)	Pale lavender white	`#F4EFFC`	Just barely purple-tinted
Water (lakes, rivers)	Medium purple	`#7B4FA6`	Strong but not distracting
Ocean / large water	Deep purple	`#4D1979`	TCU Dark Purple
Major roads	Dark charcoal	`#3C3C3C`	Roads as dark lines against light land
Minor roads / streets	Medium grey	`#9C9C9C`	De-emphasized
Motorways / highways	White	`#FFFFFF`	White on dark purple for contrast
Land use (parks)	Very light purple-green	`#D8D0E8`	Muted, not green-green
Urban areas	Light grey-white	`#EBEBF0`	Slightly darker than base land

Element	Color Name	Hex Code	Notes
Place name labels	TCU Deep Purple	#4D1979	For city/town labels
Road labels	White	#FFFFFF	Labels on darker road lines
Red accent (use sparingly)	TCU Accent Red	#C8102E	Reserve for 1 or 2 elements only

> **Design principle:** Less is more with accent colors. The red should appear in at most one place — perhaps as the color for your POI (point of interest) labels. Using it for roads, water, AND labels would be overwhelming.

Step-by-Step Instructions

Create a New Style:

1. Go to https://studio.mapbox.com and sign in
2. On your dashboard, click **"+ New style"**
3. Select the **"Blank"** template — do not use a pre-made style. Starting blank ensures every visual decision is intentional.
4. Click **"Customize Blank"**
5. The style editor opens with an empty dark canvas

Add Land, Water, & Sky:

6. Click **"+ Add component"** in the left panel
7. Select **"Land, water, & sky"**
8. Expand the component settings and set:
    - **Land color**: #F4EFFC
    - **Water color**: #7B4FA6
    - **Ocean / sea color**: #4D1979

Add the Road Network:

9. Click **"+ Add component"** → **"Road network"**
10. Style each road tier:
    - **Motorway**: White #FFFFFF (will show brightly as a major artery)
    - **Primary road**: Light grey #CCCCCC
    - **Secondary / local streets**: Medium grey #9C9C9C
11. Toggle road labels ON, colored white #FFFFFF

Add Place Labels:

12. Click **"+ Add component"** → **"Place labels"**
13. Set label color to TCU Deep Purple #4D1979
14. Choose a clean sans-serif font — **"Open Sans"** or **"Roboto"** read well on purple-tinted backgrounds
15. Set country and state labels slightly larger; city and town labels at normal size

Add Land Use:

16. Click **"+ Add component"** → **"Land use"** (or **"Parks & recreation"**)

17. Set parks/green areas to `#D8D0E8` — a very muted, slightly-purple grey-green

Add POI Labels (Your One Chance for Red):

18. Click **"+ Add component"** → **"POI labels"**
19. This is where you may use the red accent: set the POI label color to `#C8102E`
20. Keep POI icon sizes small so they do not overwhelm the map

Adjust Zoom-Level Styling:

21. Click on your **Road network** component
22. Look for **"Style across zoom range"** — set road widths to grow slightly as the user zooms in
23. For **Place labels**: set labels below zoom 7 to be invisible (too crowded at small scales), and increase label size slightly above zoom 12

Publish and Get Your Style URL:

24. Click **"Publish"** (top right of Mapbox Studio)
25. Confirm by clicking **"Publish"** in the dialog
26. Click **"Share"** → set visibility to **"Public"**
27. Copy your **Style URL** — it looks like:

    ```
    mapbox://styles/yourusername/clxabcdefghij1234567890
    ```

28. Keep this URL handy for Script 3

Step 5: Run Script 3 — Map with Your TCU Custom Style

Part A: Add Your Credentials to Script 3

1. Open `scripts/03_map_custom_style.py`
2. Fill in both variables near the top of the file:

```
MAPBOX_TOKEN = "pk.eyJ1IjoieW91cnVzZXJuYW1lIiwiYSI6ImNsZX..."

MAPBOX_STYLE_URL = "mapbox://styles/yourusername/yourstyleid"
```

3. Save the file (**Ctrl+S**)

Part B: Run the Script

```
python scripts/03_map_custom_style.py
```

The script creates `data/fw_tacos_map_custom.html`.

Compare your two maps:

- `data/fw_tacos_map_basic.html` — plain OpenStreetMap
- `data/fw_tacos_map_custom.html` — your custom TCU-themed style

Open both in the browser and compare. Notice that **all the data, markers, and popups are identical** — only the basemap has changed. This shows how separating your data layer from your basemap style gives you design flexibility.

Connection to Lab 6

Today's Exercise	Lab 6 Equivalent
JS scraping lesson from Script 1	Understanding data acquisition challenges
Script 2 — Mapbox geocoding API	Exact same API structure for Lab 6
Folium markers with custom colors	Lab 6 requires different colors per location type
Step 4 — Mapbox Studio from blank	Lab 6 Part 1: your inspiration-based style
Script 3 — swapping the basemap tile URL	Lab 6 Part 3 Step 4: same technique

---

Troubleshooting

Problem	Solution
`ModuleNotFoundError`	Run `pip install -r requirements.txt` in the `1100` folder
Script 2 says "Could not find fw_tacos.csv"	Run Script 1 first
Map opens but the basemap is grey/blank	Check that your token and style URL are both correct
Markers show but basemap shows OpenStreetMap	You may have used the wrong URL format — use `mapbox://styles/...` not the tile URL
Geocoding returns 0 results	Verify your token is correct and your internet is working